FULL-DAY WORKSHOP

# Successful Semantic Modelling for Power BI

Design, scale, and validate enterprise-grade semantic models, then make them Copilot-ready.

**ATTENDEE GUIDE**

**Phil Seamark & David Browne**

Microsoft Fabric Customer Advisory Team

First delivery: DataCon, Seattle · August 2026

Draft v0.1 · 26 June 2026

# How to use this guide

This guide is yours to keep. It follows the same order as the day, so you can read ahead, take notes in the margins, and re-run every lab afterwards on your own machine. Each module opens with a divider page, then walks through the concepts, and ends with a hands-on lab you can follow step by step.

Look for these markers throughout the day:

**Lab**
A hands-on exercise. Follow the numbered steps. Everything you need runs in a browser.

**Tip**
A shortcut or good-practice nudge worth remembering.

**Watch out**
A common trap that bites people in real models.

**Key message**
The one idea to take away from a section.

**TIP** **YOU ONLY NEED A BROWSER**

We provide the data and the starting models. Claim a numbered user, create your own workspace, copy the `00-setup` notebook into it and run it, then run `0-create-lab-models`. Your environment is then ready, on any operating system.

# Your day at a glance

| Time (PDT) | Module | You | What you will do |
|---|---|---|---|
| 9:00 - 9:20 | Welcome, setup, and lab bootstrap | **Hands-on** | Get your own workspace and starting assets |
| 9:20 - 10:05 | 1. Modelling fundamentals that drive performance | **Hands-on** | Star schema, grain, facts, dimensions, relationships |
| 10:05 - 10:50 | 2. Storage mode strategy | Watch + exercise | Import vs Direct Lake vs DirectQuery vs Hybrid |
| 10:50 - 11:05 | Break | | |
| 11:05 - 11:45 | 3. RLS design done right | **Hands-on** | Dimension-based security, static vs dynamic roles |
| 11:45 - 12:35 | 4. Scaling techniques | Watch + predict | Aggregations, hybrid tables, partitioning, column splitting |
| 12:35 - 1:30 | Lunch | | |
| 1:30 - 2:15 | 5. File health, size, and sorting | Watch | V-Order, sorting, Parquet shape for Import + Direct Lake |
| 2:15 - 3:05 | 6. Common DAX patterns + the new Calendar feature | **Hands-on** | Period comparison, running total, ratios, ranking, distinct count |
| 3:05 - 3:20 | Break | | |
| 3:20 - 4:05 | 7. Diagnosing slow visuals | **Hands-on** | A repeatable triage and tuning workflow |
| 4:05 - 4:45 | 8. Load testing and monitoring | Demo | Install and run a real load test; Delta Analyzer, BPA, Capacity Metrics |
| 4:45 - 5:00 | 9. Making your model Copilot-ready | **Hands-on** | Names, descriptions, synonyms, a Copilot answer |

Timings are a guide. The **You** column tells you when to have your laptop open. Modules 4 and 5 are watch-only, along with parts of Modules 8 and 9, because they need tooling or a capacity we cannot hand out. In both watch-only modules you will still be asked to *predict* the result before it is revealed, so stay awake. Everything else is yours to do.

# Goals for the day

Here is what we want you to walk out with. These are deliberately modest and practical, not a promise to turn you into a performance guru in a day. If you can tick most of these by 5:00, the day was worth your time.

**BY THE END OF TODAY YOU SHOULD BE ABLE TO**

- Look at a model and name the handful of design choices that make it fast or slow
- Choose a storage mode, and say in one sentence why you chose it
- Add a simple row-level security role and check it with View as role
- Pick one scaling technique that fits a real problem, rather than guessing
- Prove a change made a model faster, instead of just feeling that it did
- Leave with a reusable checklist you can apply to your very next model

**TIP** **KEEP THIS PAGE OPEN**

We come back to these goals at the end of the day. As each one clicks into place, tick it off. It is a quick way to see the day adding up.

> *Small, concrete goals you can actually use tomorrow beat an ambitious list you forget by Friday.*

MODULE 1

# Modelling fundamentals that drive performance

Most performance problems are model problems wearing a DAX costume. Before you tune a single measure, get the shape of the model right.

**IN THIS MODULE**

Grain · facts vs dimensions · relationships and filter flow · surrogate keys · cardinality and the traps that wreck compression · importing only what you need.

# 1. Modelling fundamentals that drive performance

**BY THE END OF THIS MODULE YOU CAN**

- State the grain of a fact table in one sentence, and explain why it matters
- Separate fact responsibilities from dimension responsibilities
- Read how a filter flows from a dimension to a fact across a relationship
- Spot the design choices that quietly destroy compression and query speed

## Start with the grain

The **grain** is the meaning of a single row in your fact table: "one row per order line", "one row per daily inventory snapshot", "one row per meter reading". Decide this first and write it down. Every measure, relationship, and filter assumption depends on it. When a model feels confusing, it is almost always because two grains have been mixed into one table.

## Facts and dimensions have different jobs

A **fact** table holds the events you measure: it is tall, narrow, numeric, and additive where possible. A **dimension** describes the context you slice by: who, what, where, when. Dimensions are short and wide; facts are long and lean. Keep descriptive text out of fact tables, keep measures out of dimensions, and the model almost designs itself.

## Relationships and filter flow

Relationships exist to carry a filter from the "one" side (a dimension) to the "many" side (a fact). Picture the arrows: select a year on the date dimension, the filter flows down to the sales fact, and your measure responds. Single-direction relationships are predictable and fast. Bidirectional relationships feel convenient but make filter flow ambiguous and expensive.

**WATCH OUT** **BIDIRECTIONAL RELATIONSHIPS**

Reach for bidirectional only when you genuinely need it (for example, a many-to-many bridge), and even then prefer to solve it in the model. A model full of two-way arrows is a model where nobody can predict the answer.

## Surrogate keys

Join facts to dimensions on a single, meaningless integer key, not on a natural business key like an email address or a product code. Integer keys compress beautifully, relationships evaluate faster, and you stay insulated from source systems renaming or reusing business codes.

## Cardinality is the hidden cost

VertiPaq compresses columns brilliantly, but its job gets harder as the number of distinct values (the cardinality) rises. High-cardinality columns, ultra-wide tables, and deep snowflakes all bloat the model and slow scans. The cheapest performance win is often simply not importing a column you never use.

**TIP** **IMPORT ONLY WHAT YOU NEED**

Every column you load costs memory and refresh time forever. If a column is not used by a relationship, a measure, or a visual, leave it in the source.

> *Most performance problems are model problems wearing a DAX costume.*

**LAB 1** **FIND FIVE DESIGN ISSUES**

Approx. 10 minutes · browser · model: `01 Baseline (messy)`

You have been handed a deliberately messy sales model. Before changing anything, your job is to **diagnose**. Open it in the browser and answer the questions, then we will fix them together.

1. In your workspace, open the `01 Baseline (messy)` semantic model and choose **Open data model** (the web model view).
2. Write down the **grain** of the `Sales_messy` table in one sentence.

   **This one is deliberately not visible in the model** — the other four are, so do not feel you have missed something obvious. It lives in the *data*, not the definition. Sampling will not find it either: the odd rows sit at the *bottom* of the table, so `EVALUATE TOPN ( 100, 'Sales_messy' )` shows you nothing but ordinary order lines. Run this instead, in the DAX query view:

   ```
   EVALUATE SUMMARIZECOLUMNS ( Sales_messy[RowType], "Rows", COUNTROWS ( Sales_messy ), "Sales", SUM ( Sales_messy[SalesAmount] ) )
   ```

   Two rows come back. Look hard at the two **Sales** figures.
3. Find one table that is acting as **both a fact and a dimension**. What gives it away?
4. Identify a **bidirectional relationship**. What ambiguity could it cause?
5. Spot a relationship built on a **natural (text) key** instead of a surrogate key.
6. Using the field list, find the **highest-cardinality column** you suspect is unused.

**Capture your five findings on the answer sheet overleaf.**

## Lab 1: your answers

One or two lines each is plenty. Question 1 is just opening the model, so there are five findings to capture. They are numbered here to match the questions.

2. The **grain** of `Sales_messy`, in one sentence

3. The table acting as **both a fact and a dimension**, and what gives it away

4. The **bidirectional relationship**, and the ambiguity it could cause

5. The relationship built on a **natural (text) key**

6. The **highest-cardinality column** you suspect is unused

**LAB 1B** **BUILD THE CLEAN STAR**

Approx. 14 minutes · browser (New semantic model) · your own lakehouse

You already have the finished reference in your workspace, `01 Star Schema (fixed)`. Now build your own version to match it. The value is in the doing, wiring the relationships, marking the date table, setting the sort-by, so the moves stick in your hands. This is not a test: if you get stuck at any step, open the reference and follow along, or simply watch. The later labs each come with their own ready-made model, so nothing here can put you behind.

1. Open **your own lakehouse** (the one with shortcuts to the shared data) and click **New semantic model** on the ribbon. Launching it from the *lakehouse* view gives you **Direct Lake over OneLake** (from the SQL endpoint view you would get DL/SQL).
2. Name it `My Star Schema` (so it sits beside the reference), tick the five clean tables, `Date`, `Product`, `Customer`, `Territory`, `Sales`, and Confirm.
3. In **Open data model**, create the five relationships (all Many-to-one from `Sales`): `OrderDateKey → Date[DateKey]` (active), `ShipDateKey → Date[DateKey]` (**make it inactive**), `ProductKey`, `CustomerKey`, `TerritoryKey` to their dimensions.
4. Add `Total Sales = SUM(Sales[SalesAmount])` and mark `Date` as a date table.
5. Set `MonthYear`'s **Sort By Column** to `MonthYearSort` so it sorts chronologically.
6. Prove it works: click **New report** on the ribbon to open a report canvas in the browser, then put `Total Sales` by `Date[MonthYear]` into a column chart. You do *not* need to save the report, it is only there to prove the model behaves.

**Done looks like:** the chart shows sales by month, the months run in *calendar* order rather than alphabetically (that is your Sort By Column working), and the totals match the `01 Star Schema (fixed)` reference. If the months read "Apr, Aug, Dec...", revisit step 5. If the totals look wrong, check your relationships in step 3.

MODULE 2

# Storage mode strategy

Choose Import, Direct Lake, DirectQuery, or Hybrid by workload, with Direct Lake as the default Fabric story.

**IN THIS MODULE**

Import vs Direct Lake vs DirectQuery vs Composite · Direct Lake first · OneLake vs SQL endpoint · transcoding and column paging · DirectQuery fallback · latency, concurrency, freshness, cost · Hybrid tables · a reusable decision matrix.

# 2. Storage mode strategy

**BY THE END OF THIS MODULE YOU CAN**

- Describe how Import, Direct Lake, DirectQuery, and Composite each serve data
- Explain why Direct Lake is the default Fabric choice, and when it is not
- Recognise what triggers DirectQuery fallback and why it is not something to fear
- Choose a storage mode from an SLA, refresh window, and query shape

## Four ways to serve the same data

**Import** loads a compressed copy into memory: fastest queries, but you pay a refresh cost and hold a full copy. **DirectQuery** leaves the data at source and translates every visual into a source query: always current, but only as fast as the source. **Direct Lake** reads Delta/Parquet straight from OneLake into the VertiPaq engine on demand, giving Import-like speed without a refresh step. **Composite** mixes modes in one model, for example a DirectQuery detail table with an Import aggregation over it.

## Direct Lake first

On Fabric, start from Direct Lake and justify anything else. It gives you the query performance of Import with the freshness of reading the lake directly, and it removes the refresh window that dominates large Import models. Position the other modes as the exceptions: Import when you need full engine control or you are not on Fabric, DirectQuery when data must never be copied, Composite when one table has a genuinely different profile from the rest.

## On OneLake vs on a SQL endpoint

Direct Lake on OneLake reads the Delta tables directly and is the leanest path. Direct Lake on a SQL endpoint goes through the warehouse/SQL analytics endpoint, which adds capability but also a layer. Know which one you are on, because it changes how freshness and permissions behave.

## How Direct Lake actually serves a query

The first time a column is needed it is **transcoded** from Parquet into VertiPaq form and paged into memory; after that it behaves like an Import column. This is why a cold query can be slower than the warm queries that follow. Column-on-demand paging means you only pay for the

columns you touch, which rewards narrow, well-shaped tables.

## DirectQuery fallback is not a failure

If a Direct Lake query cannot be served from memory, for example it exceeds a capacity guardrail, the engine quietly falls back to DirectQuery for that query. The result is still correct; it is just slower. Treat fallback as a signal to check your file layout and guardrails, not as an error to panic about.

## What Hybrid tables solve

A Hybrid table keeps most history in Import partitions and the most recent slice in DirectQuery, so you get fast historical queries and near-real-time recent data in one table. It is the answer to "I need years of history and up-to-the-minute today" without importing everything constantly.

**WATCH OUT** **DIRECT LAKE CAPACITY GUARDRAILS**

Each SKU caps Parquet files per table, row groups per table, rows per table, model size on disk, and memory. What happens when you cross one depends on which Direct Lake you are on: Direct Lake on SQL falls back to DirectQuery, while Direct Lake on OneLake does not fall back at all, the refresh fails and the model stops answering until the Delta tables are fixed. Only model size on disk is evaluated model-wide; the rest are checked per query. The guardrails, not your optimism, decide when Direct Lake stays in memory.

**The current numbers, by SKU:**
learn.microsoft.com/fabric/fundamentals/direct-lake-overview#fabric-capacity-requirements

## A decision matrix you can reuse

| Workload | Freshness need | Sensible default | Why |
|---|---|---|---|
| Small, high-concurrency finance | Daily | Import (or Direct Lake) | Small enough to import; concurrency loves in-memory |
| Near-real-time operational | Minutes | Hybrid or DirectQuery on the hot slice | Recent data must be live; history can be fast |
| Very large historical, hot recent | Hourly/daily | **Import + Hybrid** | Hybrid partitions need an Import base. Direct Lake does not support hybrid tables, so a live hot slice puts you in Import |

| | | | |
|---|---|---|---|
| Very large historical, no live slice | Hourly/daily | Direct Lake | Avoids the refresh window; scales with the lake |

**EXERCISE 2** **PICK A MODE, DEFEND IT**

Approx. 5 minutes · discussion · no file needed

For each scenario below, choose a storage mode and write one sentence justifying it. We will compare answers and build a shared decision matrix.

1. A 2-million-row finance model, 400 concurrent users, refreshed overnight.
2. An operations dashboard that must show events from the last five minutes.
3. A 3-billion-row sales history where only the current quarter changes.

> *Lead with Direct Lake, size against the guardrails, and let the workload pick the exception.*

MODULE 3

# RLS design done right

Secure on a dimension and let the filter flow to the facts. Keep expressions simple. Static vs dynamic roles.

### IN THIS MODULE

Secure the dimension, not the fact · avoid `LOOKUPVALUE` on facts · keep the expression trivial · static vs dynamic roles · the mapping-table pattern with `USERPRINCIPALNAME()` · validating with View as role.

# 3. RLS design done right

**BY THE END OF THIS MODULE YOU CAN**

- Apply row-level security on a dimension and let it flow to the facts
- Explain why RLS in a semantic model travels through relationships, unlike a database
- Explain why filtering the fact table directly is the slow, fragile path
- Choose between a static role and a dynamic mapping-table role, and build both
- Validate what a user can see with View as role, and know who RLS does not apply to

## Secure the dimension, not the fact

Put the security filter on a dimension, for example `Territory` or `Customer`, and let the existing relationship carry it down to the fact. The engine already knows how to flow a filter from the one side to the many side; RLS is just one more filter riding those rails. Filtering the fact table directly throws away that leverage and runs a heavier predicate on every query.

## RLS here travels through relationships

This is the biggest difference between security in a semantic model and security in a database, and it is worth pausing on. A relational database has no concept of a relationship the engine can follow at query time, so a security policy has to be defined on *every* table holding sensitive data. Miss one and you have a leak.

A semantic model is not like that. An RLS filter propagates along relationships exactly like any other filter. Secure `Territory` and every fact table related to it is secured too, automatically, because that is simply how filters move. One rule on one dimension can protect the whole model. That is why the advice is to secure the dimension, and it is why RLS here is usually a much smaller piece of work than people coming from a database background expect.

The flip side is that your security is only as good as your relationships. If a fact table is not related to the secured dimension, it is not protected. Orphan tables are a security problem here, not just a modelling one.

## Avoid LOOKUPVALUE on the fact table

Reaching for `LOOKUPVALUE` inside an RLS rule on the fact table is a common cause of slow, brittle security. It forces a row-by-row lookup on the largest table in the model, on every query, for every user. If you find yourself writing it, step back: there is almost always a dimension you can

secure instead.

## Keep the expression trivial, and know why

An RLS filter is not evaluated once. It gets attached to **every storage engine query that touches any table carrying the filter**, for every user, for the life of the model. That is the reason simplicity matters here more than anywhere else in the model.

Done well it can make queries *faster* than having no RLS at all, which surprises people. A simple equality filter like `[City] = "Seattle"` cuts the rows the engine has to scan, and the engine can usually resolve it with a bitmap index, which is close to free. You are handing the storage engine a cheap way to do less work.

Done badly it goes the other way, hard. Nested conditions, string manipulation and multi-table gymnastics cannot take that cheap path, so the engine evaluates the expression the expensive way on every scan, on every query, for every user. That is how a security rule quietly maxes out a capacity. Complexity in the security layer is the most expensive complexity you can add.

## The two kinds of RLS

There are two patterns, and picking the right one is most of the design decision.

**1. Static RLS.** You create several roles and write the filter rule directly on each one. A `USA` role filters `[Country] = "USA"`, a `Canada` role filters `[Country] = "Canada"`, and so on. You then add people to roles individually or, far better, by security group. Someone can belong to more than one role, in which case they see the *union*: put a person in both roles above and they see USA and Canada sales together. The rules live on the role, so the model itself carries the definition of who sees what.

It is simple, obvious to read, and perfectly good for a handful of slices. It stops being practical once you have dozens of territories, because every new slice means a new role and a model change.

**2. Dynamic RLS.** You keep *one* role, and move the decision into the data. A control table holds rows describing who can see what: one row saying Phil / USA, another saying David / Canada. The role filter is just `[UserEmail] = USERPRINCIPALNAME()`, which matches the signed-in user to their rows, and the relationship carries that through to the facts.

The model no longer knows or cares who sees what, the table does. Granting someone access is an `INSERT`, not a model change and republish, which is what makes this the pattern that scales.

**TIP** **THE DYNAMIC MAPPING PATTERN**

Create a `UserAccess` table (user email + the dimension key they may see). Relate it to the dimension. The role filter becomes simply `[UserEmail] = USERPRINCIPALNAME()`. To change access, edit a row, not the model.

**WATCH OUT** **RLS DOES NOT APPLY TO ADMINS**

RLS only bites for people with **read** access. Workspace Admins, Members and Contributors, and the owner of the model, bypass it entirely and see everything. This catches people out constantly: you finish building RLS, open the report yourself, see all the data and conclude it is broken. It is not, you are just an admin.

The way to check is to test as someone else. In Power BI Desktop that is **View as**, where you can tick a role *and* enter an **Other user** at the same time. The service works differently: on the semantic model's **Security** page, **Test as role** lets you pick a role *or* name a person, never both. Naming a person applies *every* role that person belongs to, so if they sit in three roles you see the union of all three. Testing as a user is still the only way to prove a *dynamic* rule, because it exercises the control table lookup rather than just the role definition, but choose an account whose role membership you actually know.

**TIP** **DEFINE IT ONCE, IN ONELAKE**

If your model is **Direct Lake on OneLake**, rules defined with **OneLake security** propagate through to the semantic model, so you do not have to define them a second time in the model at all. The same rules also apply to everything else reading that data, Spark, the SQL endpoint, other models, which is the real prize: one definition enforced everywhere, rather than one per engine and the drift that comes with it.

Note this is specific to Direct Lake *on OneLake*. Direct Lake on a SQL endpoint and Import both still need the rules defined in the model as normal, which is one more practical consequence of the Module 2 distinction. Check where the feature has got to on your tenant before you plan around it.

**WATCH OUT** **BIDIRECTIONAL FILTERS AND RLS**

RLS plus bidirectional relationships is where surprising results and slow queries breed. Keep relationships single-direction where you can, and test every role. The one deliberate exception is the mapping table itself: it sits on the *many* side, so its relationship to the dimension must be **bidirectional** with **Apply security filter in both directions** on, or the role filters the mapping table and stops there, and everyone quietly sees everything. The engine will not accept the security setting on a single-direction relationship, so the two travel together.

> *Secure on the dimension, keep the expression trivial, and let relationships do the work.*

**BEFORE LAB 3** **TWO THINGS GATE "TEST AS ROLE"**

Neither is obvious, and each one fails with a message that points somewhere else. Doing only one of them still fails. Get both out of the way before you write any roles.

**1. A report has to exist in the workspace.** Role testing renders through one, and `03 RLS - Territory and Category` is already there for you. Without any report you get "*To test row-level security, create a report with the semantic model and save it to the same workspace*", which reads like a permissions problem and is not.

If you cannot see the report, **refresh the browser**. The workspace list is cached in the page, so anything the setup notebook created while you had the tab open stays invisible until you reload. It is there, you just cannot see it yet.

**2. The model needs a stored credential.** Direct Lake on OneLake reads the files as *you*, and role testing cannot impersonate somebody else through SSO, so it refuses with "*Test as role does not work with Single Sign-On (SSO)*". Do this once, on your own `03 RLS` model:

1. Find `03 RLS` in your workspace. Click **...** (More options), then **Settings**.
2. Open the **Gateway and cloud connections** section. Under **Cloud connections**, choose **Create a connection** from the dropdown. The source details load for you.
3. In the **New connection** panel, name it `Workshop <your login>`, for example `Workshop SSM.user17`. **Connection names have to be unique across the entire tenant**, not just your own account, so a shared name works for whoever gets there first and everyone else gets "*Duplicate connection name*". Your login cannot clash with anyone else's.
4. Set **Authentication method** to **OAuth 2.0**, click **Edit credential** and sign in. Leave every other setting at its default.
5. Click **Create**.
6. Back in **Gateway and cloud connections**, select your connection from the dropdown and click **Apply**.
7. **Refresh the model. Do not skip this one.** In the workspace, click the refresh icon on `03 RLS`, or **...** > **Refresh now**. Binding a credential does *not* reframe the model, and until it reframes under that new credential **"Test as role" keeps failing**. It takes seconds and copies no data.

The warning banner at the top of the Settings page disappears once the connection is applied. **That banner is not the finish line.** It tells you the credential is bound, nothing more. Refresh, then test.

**LAB 3** **ADD STATIC AND DYNAMIC ROLES**

Approx. 10 minutes · browser · model: `03 RLS`

1. Open the `03 RLS` model and choose **Open data model**. Under **Manage roles**, create a static role `Seattle Customers` on the `Customer` table: `[City] = "Seattle"`. Then create a second one, `Paris Customers`, identical apart from `[City] = "Paris"`.
2. Use **View as role** on each in turn. The City column collapses to that one city, but notice all eight regions survive: those customers buy from stores everywhere, so securing `Customer` does not narrow `Territory`. The map keeps all eight bubbles, they just shrink.
3. Now look at what you just wrote: the same role twice, one word different. `Customer[City]` holds ten cities, so this pattern needs ten roles, and a new city means editing the model. Hold that thought.
4. Create a second role `Dynamic Territory` on the `UserAccess` table. **Switch the dialog to the DAX editor first.** The default editor wraps `USERPRINCIPALNAME()` in quotes, which turns it into a literal string and the role then matches nobody. The filter is `[UserEmail] = USERPRINCIPALNAME()`.
5. Use **Test as role** and pick `Dynamic Territory`. It resolves `USERPRINCIPALNAME()` to *you*, and your own login is granted two regions, so two regions is what you should see.
   - Seeing *every* region? The security filter on the `UserAccess` relationship is not flowing both ways.
   - Seeing *nothing*? Go back to step 4 and check for quotes around `USERPRINCIPALNAME()`.
6. Assign members on the model's **Security** page, the same page as **Test as role**. Creating a role does not give anyone access; membership is a separate step, and a role with no members secures nobody in production. **You do not need membership for *Test as role*** — that previews any role whether or not anyone is in it. You need it for the next two steps, where you test as a **person** and pick up every role that person belongs to. Assign these three:
   - `Seattle Customers` → `SSM.user13@fabricconf.onmicrosoft.com`
   - `Paris Customers` → `SSM.user2@fabricconf.onmicrosoft.com`
   - `Dynamic Territory` → `SSM Capacity Group`, and nothing else
7. Now test as a *person* instead: enter `SSM.user15@fabricconf.onmicrosoft.com`. You should see **DE and FR**, that login's two granted regions. Note you pick a role **or** a person, never both: naming a person applies every role that person belongs to. `SSM.user15` is in the group and in neither city role, so this is a clean read of the dynamic rule on its own.
8. Now test as `SSM.user13@fabricconf.onmicrosoft.com` and watch what changes. That login is in `Seattle Customers` directly *and* in `Dynamic Territory` through the group, so you get NZ and AU *plus* every Seattle customer. **Roles are additive, not intersecting.**

Most people expect the overlap and get the union. In production this is how stacking roles quietly widens access instead of narrowing it, and it is worth carrying home.

9. Look at what membership took. The two static roles needed a person named per role, so ten cities would mean ten roles and ten membership lists to keep current. `Dynamic Territory` took one group and covers all 99 logins, because `UserAccess` decides what each person sees rather than the role definition.
10. Note the difference: to change who sees what, you edited a *row*, not the model.

**Role or person, not both:** the service makes you choose. Power BI Desktop is different, its **View as** dialog lets you tick a role *and* an **Other user** together, which is how you isolate a single role's behaviour for a specific person. In the service, naming a person always applies every role they belong to, so pick your test account deliberately.

**Why a real account and not `someone@example.com`:** role testing only accepts users that actually exist in the tenant. A made-up address tests as nobody and returns an empty report, which looks like a broken role rather than a bad test.

**Why the static roles are on `Customer` and not `Territory`:** `Territory` already has a bidirectional *security* filter from `UserAccess`, and the engine will not let you add a table filter to a table that has one. You get "*To set a table filter for this role, you need to turn off the cross-filter security first*". Worth knowing before you meet it in production: a table can carry a bidirectional security filter or its own role filter, not both.

MODULE 4

# Scaling techniques

Aggregations, hybrid tables, split columns, and partitioning, framed as responses to specific pain.

**IN THIS MODULE**

Match the pattern to the pain · user-defined aggregations · hybrid tables for hot/cold · incremental refresh and partitions · splitting date and time columns · Direct Lake equivalents: V-Order, file layout, guardrails.

# 4. Scaling techniques

**BY THE END OF THIS MODULE YOU CAN**

- Pick the right scaling pattern from the symptom you are seeing
- Add a user-defined aggregation and confirm queries hit it
- Use hybrid tables and partitioning to cut refresh and query cost
- Shape columns so they compress instead of bloat

## Match the pattern to the pain

Scaling is not one trick; it is a small toolbox where each tool answers a specific symptom.

| Symptom | Reach for |
|---|---|
| Queries scan far more detail than the visual needs | User-defined aggregations |
| Refresh window is too long | Incremental refresh and partitioning |
| Need history fast and recent live | Hybrid tables |
| A single wide column wrecks compression | Split columns / reshape |

## User-defined aggregations

An aggregation table pre-summarises the fact at a coarser grain, for example daily totals per product. When a visual asks for something the aggregation can answer, the engine silently serves it from the small table instead of scanning the huge one. Detail stays available for drill-through; the common questions just get cheaper.

## Hybrid tables for hot and cold

Store cold history in Import partitions and the hot recent slice in DirectQuery. You keep fast queries over years of data while today stays near-real-time, all inside one table the report author never has to think about.

The part that makes it pay off is the **data coverage definition**, an expression on the DirectQuery partition that tells the engine which rows that partition holds. With it in place, a query that cannot possibly need the DirectQuery half skips it entirely and runs at Import speed. One catch worth remembering: a range expression like `< 20260101` has to sit on a **dual** table, so you point it at the date dimension rather than the fact. The engine needs an in-memory copy it can check cheaply before deciding whether to reach out to the source.

## Incremental refresh and partitions

Partition the fact by date and only refresh the partitions that changed. A model that used to reload everything nightly can instead refresh just the last few days, turning an hour into minutes and taking load off the source.

## Split columns to help compression

A single high-cardinality `datetime` column can be the most expensive column in the model. Split it into a `date` and a `time` column and each compresses far better, because each has far fewer distinct values. The same idea applies to composite keys and concatenated text.

## The Direct Lake equivalents

In Direct Lake you do not refresh, so scaling shifts to the file layer: V-Order and sorting on load, sensible Parquet file and rowgroup sizes, and staying inside the capacity guardrails. Module 5 goes deep on this; the mindset is the same, the levers just move upstream into the lake.

**TIP** **CHANGE ONE LEVER, THEN MEASURE**

Each of these can help or hide a problem. Add one, re-measure with the Lab 7 notebook, and keep it only if the numbers moved. Stacking three changes blind is how models get complicated without getting faster.

> *Every scaling technique is an answer to a specific pain. Name the pain first.*

**WATCH** **FOUR LEVERS, FOUR PAINS**

Approx. 45 minutes · laptops can stay shut · presenter demo

This module is a demo. Configuring a composite model with aggregations needs tooling we are not asking you to install, so your presenter drives and you predict. **Before each reveal, write down the improvement you expect.** Then see how close you were.

1. **Aggregation, hit.** A query grouped by date and product answered from a table about seven times smaller than the detail fact. Watch the storage-engine time, not the total.
2. **Aggregation, miss.** The same query grouped by *customer*. The aggregation has no customer column, so it cannot help and the query falls through to the detail fact. This is the half people forget.
3. **Hybrid and partitioning.** The same filter, written two ways. One prunes the DirectQuery partition, one does not, and the difference is entirely in how the filter was written.
4. **Column splitting.** One high-cardinality decimal column against the same values split in two. Same answer, a fraction of the memory.

Ask for the numbers if you want them written down. Everything shown here is reproducible on the models you already built.

MODULE 5

# File health, size, and sorting

Get Delta and Parquet files in shape: V-Order, sorting, and rowgroup sizing for Import and Direct Lake.

**IN THIS MODULE**

Why file layout matters in both modes · what Import resolves for you · why Direct Lake needs deliberate care · V-Order · sorting on load · rowgroup and Parquet file sizing · OPTIMIZE and V-Order maintenance.

# 5. File health, size, and sorting

**BY THE END OF THIS MODULE YOU CAN**

- Explain why file layout affects both Import and Direct Lake, differently
- Say what V-Order actually buys you, and on which columns it buys nothing
- Recognise Parquet files and rowgroups that are too small or too large
- Choose between compaction, V-Order and sorting instead of reaching for all three

## Why file layout matters

VertiPaq loves data that arrives sorted and well laid out, because similar values sit together and compress harder. In Import you pay this cost once at refresh. In Direct Lake the file layout *is* the model: the engine reads the Parquet you landed, so a messy table is slow on every single query.

## Import forgives, Direct Lake remembers

Import re-encodes and re-sorts the data as it loads, so it can quietly rescue a poorly organised source. Direct Lake does not get that second chance. Whatever shape the Delta table is in when you land it is the shape every query has to live with. That is the single biggest mindset shift when you move to Direct Lake.

## V-Order

V-Order is a Microsoft optimisation that rewrites Parquet into the row order VertiPaq wants, so columns transcode faster the first time they are touched. Turn it on for the tables your semantic model reads.

What it is *not* is a reliable way to make a table smaller. On the 100M-row fact used in this workshop, V-Order moved total bytes by about **2 per cent**, and that small number is the sum of two much larger effects pulling against each other. Low-cardinality columns almost disappeared: Quantity and Discount each fell by essentially 100 per cent. Near-unique columns got *worse*: `OrderNumber` grew 30 per cent and `SalesKey` 9 per cent, and between them those two are about a third of the table. V-Order sorts cardinality-ascending, so a column with no repeated values has nothing for it to find, and sorting for everything else scatters it further.

The payback is on read, not on disk. Measured on two copies that differed *only* by V-Order, a query hitting the columns it had helped went from **360 ms to 157 ms**. A query hitting the columns it had not moved barely at all. If you have ever turned V-Order on and seen nothing change,

that is the reason: it can only pay you back on columns your queries actually touch.

> *Seamark's Maxim: V-Order as far downstream as possible, and as far upstream as necessary.*
> *Costs you once on every write, repays you on every read.*

Which is not everywhere. V-Order takes longer and costs more compute to write, so in a layered ETL it is worth asking what the read/write ratio actually is at each hop:

| Layer | Written | Read by | V-Order? |
|---|---|---|---|
| **Bronze** / Load | Once | Silver, once | No |
| **Silver** / Stage | Once | Gold, once | Rarely |
| **Gold** / Persist | Once | Every report render and every DAX query, by every user, all day | **Always** |

Bronze and Silver are typically written once and read once by the next stage, so the extra write cost is never earned back. Gold is read thousands of times, so it always is. If you take one setting home from this module, it is this one.

Deliberately the mirror image of **Roche's Maxim** from Matthew Roche, "data should be transformed as far upstream as possible, and as far downstream as necessary". Same sentence, opposite direction: transformation belongs upstream, V-Order belongs downstream. And "as far upstream as necessary" earns its place, because a Silver table read by twenty Gold builds has flipped its own ratio and should be V-Ordered too.

## Sort on the way in, or V-Order, but not both

Sorting by the column you most often filter on before you write the Delta table gives Parquet's rowgroup statistics something useful to prune with, so the engine can skip whole rowgroups it does not need. That is real, and it is a different benefit from compression.

The catch is that V-Order re-sorts the table by its own cardinality-ascending heuristic, so if you do both, V-Order wins and your sort is gone. While this workshop was being built, pre-sorting by date compressed the date key down to 21 KB, and turning V-Order on afterwards took the same column back up to 2 MB. Decide which one you are doing, and do not pay for the other.

## Rowgroup and file sizing, and the trade nobody mentions

Too many tiny Parquet files means overhead per file and, in Direct Lake, pressure against the files-per-table guardrail. This is the lever with the biggest measured effect in the whole module: compacting the workshop's fact table from **400 files down to 24** took it from 2,934 MB to 2,401

MB, about 18 per cent, and cut the cold load into the model from roughly three and a half minutes to under one. Nothing else here comes close to that.

But bigger is not simply better. The same table written as **6 very large files** was the smallest on disk and the fastest to load, and the *slowest* to query, because a rowgroup is the unit the storage engine scans in parallel and there were only six of them. Load time and scan speed pull in opposite directions, so file sizing is a trade to be made deliberately rather than a dial to turn up. V-Order is the one lever in this module that improves query speed without costing you something else.

## Maintenance is two jobs, not one

Delta tables drift as they are appended to. Run `OPTIMIZE` on a schedule to compact small files and keep the layout clean, the same way you would rebuild an index. A table that was healthy last month may need a tidy this month.

Then remember the second job: `OPTIMIZE` does not delete the old files, it just stops referencing them. Your file count drops immediately but the storage does not come back until `VACUUM` runs. People are often surprised by a bill that did not move after a big tidy, and this is why.

**WATCH OUT** **SMALL FILES COST YOU LONG BEFORE THE GUARDRAIL DOES**

Direct Lake caps Parquet files per table by SKU, but that cap is generous: 1,000 files on F2 to F32, 5,000 on F64 and above. You will feel a trickle of tiny append files as slower loading and thinner row groups long before you are anywhere near it, so treat file count as a performance signal rather than a limit to creep up on. Compaction is the fix for both.

> *In Direct Lake the file layout is the model. Compaction is the big lever; V-Order is the one that pays you back on every query.*

**PREDICT, THEN WATCH** **THE SAME 100M ROWS, FOUR WAYS**

Your presenter will show one 100M-row fact table written four ways: as CSV, as plain Parquet with dictionary encoding and compression switched off, as Delta, and as Delta with V-Order. Before any result goes up, write down three predictions: will the **size on disk** go up or down, will the **load time** go up or down, and will the **query time** go up or down, and by roughly how much. Then check your calls. Size is the one that catches most people. The last two columns are the pair worth watching: they differ by nothing but V-Order, so any gap between them is V-Order and nothing else. You do not need to type; the point is to commit to a number first.

MODULE 6

# Common DAX patterns + the new Calendar feature

Five patterns you will write again and again, each two or three lines because the model already did the work.

**IN THIS MODULE**

Period comparison · running totals · child-to-parent ratios · ranking · distinct count as a modelling decision · the new Calendar feature · calculation groups and field parameters.

# 6. Common DAX patterns + the new Calendar feature

**BY THE END OF THIS MODULE YOU CAN**

- Write all five patterns from memory: period comparison, running total, child-to-parent ratio, ranking, distinct count
- Set up a date table so period comparisons are simple and fast
- Use the new Calendar feature to define a fiscal year once, in the model, instead of in every measure
- Treat distinct count as a modelling decision, not just a formula

## Period comparisons start in the model

Prior year, prior month, and moving averages all get easy when you have a proper date table and clean relationships. Where you need more than one date context, for example order date and ship date, use role-playing dates with inactive relationships and activate them per measure rather than duplicating the fact. Row-based time intelligence, where the comparison is precomputed as a column, can turn an expensive filter into a simple lookup.

## Running totals

Year-to-date and running totals have a helper form and a general form, and it is worth knowing both. `DATESYTD` is short and reads well, but it only works because the date table is marked as one. The general form, filtering `'Date'[Date] ≤ MAX('Date'[Date])` with `ALL('Date')` removing the current filter, is the mechanism underneath every time intelligence function you will ever use. Learn the second and the first stops being magic.

## Child-to-parent ratios

"This product as a share of its category" is the classic ratio pattern. Compute the parent total with the right filter removed (typically with `ALLEXCEPT` or a targeted `REMOVEFILTERS`) and divide with `DIVIDE` so a zero denominator is handled cleanly. Reach for `ALL` instead of `ALLEXCEPT` and the denominator quietly becomes the whole model, which is the mistake people actually make.

## Ranking

Ranking measures (top N products, league tables) are easy to write and easy to write badly. Keep the ranked set as small as the question allows, and rank over the dimension *column you are displaying* rather than the table, so the engine sorts one list instead of every combination of the

columns in it.

## Distinct count is a modelling decision

Distinct count is one of the more expensive operations because the engine cannot simply add values up. There are three ways to ask and they are not interchangeable. `DISTINCTCOUNT` is the honest answer. `COUNTROWS(VALUES( ... ))` is *not* the cheaper alternative people assume: the engine rewrites it to the same distinct count, so the plan and the cost are the same, and it only agrees when the column has no blanks. `APPROXIMATEDISTINCTCOUNT` trades exactness for speed, which is legitimate on a headline tile nobody reconciles and not legitimate on anything a finance team signs — but it is **unsupported in Direct Lake**, so it is not an option on the models you build today. If a distinct count is central to your report, design for it: the way to make it cheap is to stop asking for it, by counting the rows of a table that already sits at the grain you are counting. That is a modelling change, and it will do more for you than any clever measure.

## The new Calendar feature

The Calendar feature lets you name a calendar on your date table and tag which columns mean year, quarter, month and day. A table can carry more than one, so Gregorian and fiscal live side by side. You then reference a calendar like a table: `TOTALYTD([Total Sales], 'Fiscal')` replaces `TOTALYTD([Total Sales], 'Date'[Date], "6/30")`.

**This is a semantics feature, not a performance one.** Existing time intelligence returns identical results and keeps working untouched, so there is nothing to migrate and no speed-up to expect. What you gain is that your fiscal year is defined once, in the model, under a name, instead of as a date string retyped into every measure by whoever remembered it. It coexists happily with calculation groups.

**TIP** **CALCULATION GROUPS AND FIELD PARAMETERS**

Use calculation groups to apply one set of time calculations across many measures without copy-paste, and field parameters to let users swap measures or dimensions in a visual. Both cut the number of near-identical measures you maintain.

> *Fast DAX starts with predictable filter propagation. Shape the model, then the measure gets simple.*

**LAB 6** **MODEL FIRST, THEN MEASURE**

Approx. 15 minutes · browser (web modeling) · model: `06 DAX + Calendar`

1. Open the `06 DAX + Calendar` model and choose **Open data model**. Confirm the date table is marked as a date table.
2. Add **Sales LY** and **Sales YoY %**, then **Sales YTD** and the general-form **Sales Running Total**. Copy them from the snippets sheet.
3. Add **Share of Category** ( `DIVIDE` + `ALLEXCEPT` ) and **Product Rank**. Note how little DAX each one took, and why.
4. Add the three distinct-count measures and compare their answers on `Sales[OrderNumber]` .
5. Open the **TMDL view** and read the two calendars already defined on `Date` . Work out why `MonthYearSort` is the `month` primary and `Month` is not.
6. Run `TOTALYTD` against `'Gregorian'` and then `'Fiscal'` , grouped by month, and watch the two running totals reset six months apart.

MODULE 7

# Diagnosing slow visuals

A repeatable triage workflow: is it the visual, the DAX, the model, or the source?

**IN THIS MODULE**

A repeatable triage workflow · a trace notebook (storage vs formula engine) · front-end vs storage-engine bound · instructor tools: DAX Studio, Delta Analyzer, BPA · finding the cheapest fix that actually matters.

# 7. Diagnosing slow visuals

**BY THE END OF THIS MODULE YOU CAN**

- Follow a repeatable order when a report is slow, instead of guessing
- Read a Server Timings capture to see if a query is FE-bound or SE-bound
- Use the right tool for each layer: visual, DAX, model, source
- Choose the cheapest fix that moves the number that matters

## Triage in a fixed order

When something is slow, resist the urge to rewrite the first measure you see. Work top down:

1. **The visual.** Is it asking for too much, a giant table, dozens of measures, a huge matrix?
2. **The DAX.** Capture the query. Is one measure doing the damage?
3. **The model.** Bad relationships, high cardinality, or a missing aggregation?
4. **The source.** For DirectQuery / Direct Lake, is the slowness downstream in the lake or database?

## Read the split: front end vs storage engine

Server Timings breaks a query into **storage engine** (SE, scanning and aggregating data) and **formula engine** (FE, the single-threaded step that stitches results together). SE-bound usually means you are scanning too much: fix the model, add an aggregation, cut cardinality. FE-bound usually means the measure logic is expensive: simplify the DAX. The split tells you which half of the problem to work on.

## Your tool today: a trace notebook

Everyone can diagnose in the browser: a **Fabric notebook** runs a query, traces it, and prints the **SE/FE split**, the same information Server Timings shows, with nothing to install. Both analyzers run in a notebook too, as `labs.vertipaq_analyzer()` and `labs.delta_analyzer()`, each rendering a proper interactive report. Your presenter will also demo the desktop tools, **DAX Studio** for a richer split and **BPA** for design smells. You do not need them today, but it is worth knowing they exist.

**TIP** **WHICH ANALYZER, AND WHEN**

**VertiPaq Analyzer** describes the model *in memory*: structure, cardinality, relationships, and on Import the column sizes too. **Delta Analyzer** describes the data *on disk*: Parquet files, rowgroups and how well it is laid out. On **Direct Lake** the two split apart, because memory only holds the columns that have been paged in, and the size columns are not reported. So on Direct Lake reach for **Delta Analyzer** for anything about size or footprint, and VertiPaq Analyzer for everything about shape. Every model today is Direct Lake, so both get used, for different questions.

**WATCH OUT** **TUNING THE WRONG LAYER**

Rewriting DAX on an SE-bound query, or reshaping the model on an FE-bound one, burns time and moves nothing. Read the split first, then act.

**WATCH OUT** **DO NOT PRESS "REFRESH DATA AND SCHEMA"**

In the web model editor that button also runs **auto-detect relationships**, which overwrites the relationships your setup notebook built. Your model is deliberately mis-shaped for this module, so auto-detect will helpfully "fix" the very thing you are here to find, without telling you. Nothing today needs it. If you press it by accident, say so rather than repairing it by hand.

*Diagnose in order, read the SE/FE split, then spend your effort where the time actually is.*

**LAB 7** **CLASSIFY THREE SLOW QUERIES**

Approx. 15 minutes · browser (notebook) · model: `07 Slow Visual Triage`

1. Open the `lab07-diagnose-slow-visuals` notebook and point it at your `07 Slow Visual Triage` model.
2. Run the three query cells. The notebook traces each and prints the **SE/FE split**.
3. Classify each as **SE-bound** or **FE-bound**.
4. Match each to its root cause: a bad model, a bad measure, or a cardinality/compression problem.
5. Apply the cheapest fix for one in the web model, then re-run its cell. Did the split shift as you expected?

MODULE 8

# Load testing and monitoring

One user proves nothing. Test at concurrency, then watch the model in production.

**IN THIS MODULE**

Why concurrency is a different question · installing and running a real DAX load test · reading duration, QPS, active users and engine CPU together · monitoring with Delta Analyzer, BPA, Log Analytics and Capacity Metrics · a lightweight operating cadence.

# 8. Load testing and monitoring

**BY THE END OF THIS MODULE YOU CAN**

- Explain why single-user timings say nothing about concurrency
- Install and run a real DAX load test against a model of your own
- Read query duration, QPS, active users and engine CPU together
- Set up basic ongoing monitoring for a production model

## One user proves nothing

Everything up to here measured a *single* query on a quiet capacity. That is the right way to diagnose and the wrong way to predict. A model that answers in 400 ms for you can fall over at a hundred users, because memory, CPU and the query queue are all shared. Concurrency is a different question and it needs a different test.

## Load testing without starting a project

Watch this one rather than following along. **FabricDaxLoadTest** simulates many concurrent users running your real DAX against the XMLA endpoint, then charts query duration, queries per second, active users and engine CPU. Setup is a single notebook: import it, drop a **Performance Analyzer** export onto the Resources panel, edit one cell, Run All. Set `LAKEHOUSE_NAME = None` and it does not even need a lakehouse.

The queries come from Performance Analyzer, which now runs in the **browser** as well as in Power BI Desktop, so capturing a realistic workload needs nothing installed. Take it away and point it at a model you actually care about: github.com/dbrownems/FabricDaxLoadTest.

## Concurrency and repeatability

A query that is quick for one user can fall over under a hundred. Test at a realistic concurrency, and run each test more than once, because a single run can be lucky or unlucky. Repeatable numbers are the ones you can defend.

## Monitoring once it is live

**Delta Analyzer** (or **VertiPaq Analyzer** on an Import model) and **BPA** keep the model design honest. **Log Analytics** and the **Capacity Metrics** app show how it behaves in production: which queries are slow, when the capacity is under pressure, and whether things are drifting. Monitoring

turns a one-off tune-up into an ongoing habit.

**TIP** **A LIGHTWEIGHT CADENCE**

You do not need a heavy process. A monthly glance at Capacity Metrics, a BPA pass after any significant model change, and a saved baseline query per key report will catch most regressions before your users do.

> *A model that is fast for one user has told you nothing about a hundred.*

MODULE 9

# Making your model Copilot-ready

Friendly names, descriptions, and synonyms so both humans and Copilot get good answers.

**IN THIS MODULE**

Friendly names · descriptions · synonyms · hiding the junk · logical grouping · verified answers and the linguistic schema · why good modelling helps humans and AI alike.

# 9. Making your model Copilot-ready

**BY THE END OF THIS MODULE YOU CAN**

- Name and describe objects so a person or Copilot can understand them
- Add synonyms so real-world wording maps to your fields
- Hide the technical clutter that confuses both humans and AI
- Run your model through a final "is it ready?" checklist

## Why an agent needs this

An agent never sees your data. It sees your **model definition** — table names, column names, measure names, descriptions, relationships — and from that it has to turn a human sentence into DAX. That metadata is its entire world.

So every ambiguity left in the model is a fork in the road where it can go the wrong way. Forty visible columns where six would do is thirty-four extra wrong turns. A measure called `Margin` with no description could be currency or a percentage. `fct_sls_amt` is a guess. None of these stop it answering. They just make the answer less likely to be the one you wanted.

**And an agent fails differently from a person.** A new starter who cannot work out which field to use comes and asks. An agent returns a confident, fluent, plausible answer built on the wrong column. The ambiguity never surfaces as a question — it surfaces as a number, in front of someone who trusts it.

| What you do | What it takes away from the agent |
|---|---|
| Hide keys and helper columns | Wrong choices. The cheapest win of the lot: an object it cannot see is an object it cannot pick. |
| Business-friendly names | Guesswork. Names are the first thing it matches a question against. |
| Descriptions on measures | Uncertainty about *when* to use something. The name says what it is; the description says when it applies. |
| Synonyms | Vocabulary mismatch, so "revenue", "turnover" and "takings" all land on the same measure. |
| AI instructions | Everything the model cannot say on its own: which table to trust, when the fiscal year starts, which relationship is deliberately inactive. |

## Names, descriptions, synonyms

Copilot reads your model the way a new colleague would. `Sales Amount` beats `fct_sls_amt`. A one-line **description** on a measure tells the AI what it means and when to use it. **Synonyms** map the words your business actually uses ("revenue", "turnover", "takings") onto the field, so a natural question finds the right answer.

## Hide the junk

Surrogate keys, helper columns, and staging fields should be hidden. Every object you leave visible is one more thing a person, or Copilot, can pick by mistake. A clean field list is a form of documentation.

## Group things logically

Organise measures into display folders and keep related fields together. Semantic grouping helps humans browse and gives Copilot useful structure to reason over. The tidier the model, the better the answers.

## Verified answers and the linguistic schema

For the questions that really matter, you can define **verified answers** so Copilot responds with a known-good result, and tune the **linguistic schema** so phrasing maps cleanly to your model. This is how you move from "usually right" to "reliably right" on your key questions.

## AI instructions: tell Copilot how your business talks

**Prep data for AI** on the Home ribbon opens a dialog where you write plain-English **AI instructions** for the model. Not synonyms for one field — context for the whole thing: your terminology, your busy season, which table to trust for which question. Copilot reads them before it answers.

Useful ones look like this:

- "*Revenue, turnover and takings all mean* `[Total Sales]`."
- "*Always answer with a measure. Every* `Sales` *column is hidden on purpose.*"
- "*Busy season is October to February.*"
- "*When someone asks about margin, use* `[Margin]`*, which is sales minus cost.*"

Be explicit, avoid ambiguity, and group related instructions together. A few focused instructions beat many broad ones, because conflicts confuse the model as much as they would confuse a person. They live on the *semantic model*, not the report, so every report over that model

benefits. The limit is 10,000 characters, and end users never see them.

Worth being honest about: these are guidance, not rules. The model interprets them, so there is no guarantee they are followed to the letter.

**TIP** **GOOD MODELLING WAS THE WORK**

Most of what makes a model Copilot-ready is the same work that makes it good for people: clear names, a clean star schema, sensible measures. If you did the earlier modules, you are most of the way there already.

> *A model that answers well for people answers well for Copilot. The metadata is the last mile.*

**LAB 9** **MAKE THE MODEL AGENT-READY**

Approx. 8 minutes · browser (web model view) · model: `01 Star Schema (fixed)`

1. **Rename one by hand.** `Product[ProductCode]` → `Product Code`. Nothing on `Sales` needs renaming — every column there is hidden, because the measures do that work.
2. **Let Copilot write the descriptions.** Open **Copilot** from the ribbon and ask it to add descriptions to your measures. Read what it wrote before accepting: it is a first draft from something that has only ever seen your metadata. If it guessed well, that is the metadata doing its job.
3. **Add synonyms** for one measure and one dimension — the words your business uses, not the ones in the model.
4. **Add the instructions. Prep data for AI → Add AI instructions**, paste the block from the **Module 9** section of `dax-snippets.md`, then **Apply**. Most of it just restates decisions made earlier today.
5. **Look at what it did.** Open the **TMDL view** before and after step 4. Your text lands in `Model → cultureInfo → linguisticMetadata → CustomInstructions`, and Apply generates the whole linguistic schema around it.
6. Run the end-of-day checklist: is it fast, explainable, monitorable, and agent-ready?

**Dialog refuses to open, or its tabs are greyed out?** Try turning **Q&A** on for the model — it is a separate prerequisite: **semantic model → Settings → Q&A → "Q&A to ask natural language questions about your data"**. Then open a **fresh browser tab** and retry, because the client caches this check when the session starts. If it still will not play, watch the presenter do it instead; nothing later depends on it. Writing instructions costs you nothing in capacity; only *asking* Copilot a question does.

# How did we do?

Back to where we started. This is the same list of goals from this morning. Run down it and be honest: these were meant to be achievable in a day, so most should now feel within reach. Anything still fuzzy is a great place to point your practice next week.

| This morning's goal | Can you do it now? |
|---|---|
| Name the design choices that make a model fast or slow | |
| Choose a storage mode and justify it in one sentence | |
| Add a row-level security role and check it with View as role | |
| Pick a scaling technique that fits a real problem | |
| Prove a change made a model faster | |
| Leave with a reusable checklist for your next model | |

**TIP** **ONE THING TO TRY ON MONDAY**

Pick a single model you own and apply just one idea from today: state its grain, add an aggregation, or measure a query with Server Timings. One applied idea beats a notebook full of intentions.

> *The day was worth it if you leave able to do one thing you could not do this morning.*

# Your takeaway checklist

Use this to sanity-check any model you build after today.

| Question | Yes / No |
|---|---|
| Can I state the grain of every fact table in one sentence? | |
| Is it a clean star schema, with no table doing two jobs? | |
| Are relationships single-direction on integer surrogate keys? | |
| Did I choose the storage mode deliberately, for this workload? | |
| Is RLS on a dimension, with a simple expression? | |
| Are the underlying files in good shape (sorted, V-Ordered)? | |
| Can I prove the model is fast, with a before/after measurement? | |
| Is the model explainable to a person and to Copilot? | |

> *Take a model from "works" to "scales", and prove why it scales.*